

Agent	Reward R_t = how well agent is doing at $t, t \in \mathbb{R}$. - Value fn : how good each state is and/or action. - Model : agent's representation of env	
	Stochastic policy : $\pi(a s) = \Pr(A_t = a S_t = s)$. - Deterministic : $a = \pi(s)$. Optimal policy (informal): maximizes cumulative rewards	
	Return = total (discounted) reward from t to $T = G_t = \sum_{k=0}^T \gamma^k R_{t+k+1} = R_{t+1} + \gamma G_{t+1}$, where γ = discount rate , $0 \leq \gamma \leq 1$	
	State-value fn for policy π $= v_\pi(s) = \mathbb{E}_\pi[G_t S_t = s] \forall s \in \mathcal{S}$. Action-value fn for policy π $= q_\pi(s, a) = \mathbb{E}_\pi[G_t S_t = s, A_t = a] \forall s \in \mathcal{S}$	
Markov process (MP)/ Markov chain	State S_t Markov Property iff $\Pr(S_{t+1} S_t) = \Pr(S_{t+1} S_1, \dots, S_t)$	MP = tuple $\langle \mathcal{S}, p \rangle$ and $p(s' s) = \Pr(S_{t+1} = s' S_t = s)$
	Process that ends in terminal state = episodic /episodic task. Process go continually w/o limit = continuing process/task	
Markov Reward Processes (MRPs) = MP + Rewards	MRP = $\langle \mathcal{S}, \mathcal{R}, p \rangle$, & $p(s', r s) = \Pr(S_{t+1} = s', R_{t+1} = r S_t = s)$	
	Bellman eqn $= v(s) = \sum_{s',r} p(s', r s)[r + \gamma v(s')]$ For continuing tasks, $T = \infty$, and return $= \infty$. Use avg reward OR - Set $\gamma < 1$ & $G_t \leq \sum_{k=0}^{\infty} \gamma^k R_{max} = \frac{R_{max}}{1-\gamma}$	
Markov decision process (MDPs) = MRP w actions	MDP is a tuple $\langle \mathcal{S}, \mathcal{A}, \mathcal{R}, p \rangle$, and $p(s', r s, a) = \Pr(S_{t+1} = s', R_{t+1} = r S_t = s, A_t = a)$ MDP + Policy $\Rightarrow$ MRP. Theorem : Fix any policy π . Then $p_\pi(s', r s) = \mathbb{E}_{a \sim \pi(s)}[p(s', r s, a)] = \sum_{a \in \mathcal{A}(s)} \pi(a s) p(s', r s, a)$	
Optimal Policies & Optimal Value Fns	Partial order: $\pi \geq \pi'$ iff $v_\pi(s) \geq v_{\pi'}(s) \forall s \in \mathcal{S}$. Optimal state/action-value fn $= v_*(s)/q_*(s, a) = \max_\pi v_\pi(s) / \max_\pi q_\pi(s, a)$	
	Optimal policy $= \pi_*(a s) = 1$ if $a = \operatorname{argmax}_{a \in \mathcal{A}(s)} q_*(s, a)$. $\pi_*(a s) = 0$ otherwise. Thus, $v_*(s) = \max_a q_*(s, a)$	
Bellman Optimality Eqn		$v_*(s) = \max_a \sum_{s',r} p(s', r s, a)[r + \gamma v_*(s')]$. $q_*(s, a) = \sum_{s',r} p(s', r s, a) \left[r + \gamma \max_{a'} q_*(s', a') \right]$
Policy evaluation (find v_π)	Recall Bellman expectation: $v_\pi(s) = \sum_a \pi(a s) \sum_{s',r} p_\pi(s', r s)[r + \gamma v_\pi(s')] \forall s \in \mathcal{S}$. 1) Set of linear eqns -> Solve v_π directly	
	2) Iterative: $v_{k+1}(s) = \sum_a \pi(a s) \sum_{s',r} p(s', r s, a)[r + \gamma v_k(s')] \forall s \in \mathcal{S}$. - $v_0(s)$ arbitrarily; $v_0(\text{terminal}) = 0$. - $v_{k+1}(s) - v_k(s) < \epsilon$.	
	3) Sampling: $v_\pi(s) \approx \frac{1}{N} \sum_{i=1}^N G_{t,i}$, where $G_{t,i}$ = return at timestep t on episode i (approximate by averaging returns of executing π)	
Policy improvement / Control (Pip)		Compute action-value fn q_π from (computed) state-value fn v_π : $q_\pi(s, a) = \sum_{s',r} p(s', r s, a)[r + \gamma v_\pi(s')] \forall s \in \mathcal{S}$ Greedy improvement : $\pi'(s) = \operatorname{argmax}_a q_\pi(s, a)$. Policy improvement thrm : If $q_\pi(s, \pi'(s)) \geq v_\pi(s)$, then $v_{\pi'}(s) \geq v_\pi(s)$
1) Policy Iteration (PI)	Initialize $v_{\pi_0}(s) \in \mathbb{R}$ and $\pi_0(s) \in \mathcal{A}(s)$ arbitrarily $\forall s \in \mathcal{S}$; $v_{\pi_0}(\text{terminal}) = 0$. Loop $k = 1 \dots \infty$: - Run policy eval to compute v_{k-1} - Run policy improvement: For each state s : $\pi_k(s) \leftarrow \operatorname{argmax}_a q_{\pi_{k-1}}(s, a)$. - If policy $\pi_k = \pi_{k-1}$ (stable), stop and return v_k and π_k	
2) Value Iteration (VI)	Initialize $v_0(s)$ arbitrarily; $v_0(\text{terminal}) = 0$. - $v_{k+1}(s) = \max_a \sum_{s',r} p(s', r s, a)[r + \gamma v_k(s')] \forall s \in \mathcal{S}$. - Stop when $v_{k+1}(s) - v_k(s) < \epsilon$ - If converge at step K; Output $\pi(s) \leftarrow \operatorname{argmax}_a q_K(s, a) = \operatorname{argmax}_a \sum_{s',r} p(s', r s, a)[r + \gamma v_K(s')]$. (VI = PI w 1 sweep evaluation)	

Model-free Monte Carlo (MC) Mtds – No Bootstrap	- learns from complete episodes , - only for episodic tasks , - estimates by averaging sample returns , $v_\pi(s) \approx \frac{1}{N} \sum_{i=1}^N G_{t,i}$	
	MC Prediction: first-visit and every-visit First-visit : update first time state appears Every-visit : updates all visits Theorem : Both converge to $v_\pi(s)$ as $N \rightarrow \infty$	Initialize: $\forall s \in \mathcal{S}: V(s) \in \mathbb{R}, \text{arbitrarily}$. <i>Returns</i> (s): an empty list, G = 0 Loop i = 1, ..., N: Generate an episode following π : For ea step $t = T - 1, \dots, 0$: - $G \leftarrow G + R_{t+1}$; - If S_t appears 1st time : { - Append G to <i>Returns</i> (S_t); - $V(S_t) \leftarrow \operatorname{avg}(\operatorname{Returns}(S_t))$ }
	MC Prediction for Action Values. - Issue: many state-action pairs not visited if π = deterministic (need exploration)	Same as above but use Initialize: <i>all</i> $Q(s, a) \in \mathbb{R}$ <i>arbitrarily</i> - { ... $Q(S_t, A_t) \leftarrow \operatorname{avg}(\operatorname{Returns}(S_t, A_t))$ }
	Maintaining Exploration 1) On-policy : same policy to eval & explore (near optimal) 2) Off-policy : - Target policy π (to optimize). - Behavior policy b 1) Soft policy : - $\pi(a s) > 0 \forall s \in \mathcal{S}, a \in \mathcal{A}$. 2) ϵ-soft policy : - $\pi(a s) > \frac{\epsilon}{ \mathcal{A}(s) }$ where $0 < \epsilon \leq 1$ is a param $\forall s \in \mathcal{S}, a \in \mathcal{A}$ 3) ϵ-greedy policy : - $A^* \leftarrow \operatorname{argmax}_a Q(s, a)$. - if $a = A^*$: $\pi(a s) = (1 - \epsilon) + \frac{\epsilon}{ \mathcal{A}(s) }$. Else $\pi(a s) = \frac{\epsilon}{ \mathcal{A}(s) }$	
On-Policy MC	Policy eval w MC to approx q_π , Policy improvement w ϵ -greedy Thrm : π converge to policy at least better/= other ϵ -soft policy	From MC Prediction for Action Values. Let $\pi \leftarrow$ any ϵ -soft policy. Add at end { ... - For all $a \in \mathcal{A}(S_t)$: $\pi(a S_t) \leftarrow \epsilon$ -greedy over Q }
Off-Policy MC	Importance sampling – estimate values of 1 dist given samples from another. Let $\mathbb{E}_b[G_t S_t = s] =$ expected return of policy b Expected return of policy $\pi = \mathbb{E}_\pi[G_t S_t = s] = \mathbb{E}_b[\rho_{t:T-1} G_t S_t = s]$, where $\rho_{t:T-1} = \prod_{k=1}^{T-1} \frac{\pi(A_k S_k)}{b(A_k S_k)}$ = importance sampling ratio	
Model-free Temporal difference (TD) learning – Bootstrap	- No need complete episode. - Like DP: Update before episode ends w curr estimate (bootstrap)	
	TD Prediction . TD(0) / 1-step TD: $V(S_t) \leftarrow V(S_t) + \alpha[R_{t+1} + \gamma V(S_{t+1}) - V(S_t)] = V(S_t) + \alpha \delta_t$, where $\alpha \in (0,1]$ = step-size param TD error , $\delta_t = R_{t+1} + \gamma V(S_{t+1}) - V(S_t) = [R_{t+1} + \gamma V(S_{t+1})] - V(S_t) =$ TD return (1-step estimate) - curr estimate $V(S_t)$	
	1) Online Update. Loops: W π : get (S, A, S', R) $V(S) \leftarrow V(S) + \alpha[R_{t+1} + \gamma V(S') - V(S)]$; $S \leftarrow S'$	2) Batch/Offline Update. Loops: { W π : get (S, A, S', R); Store (S, R, S') in list; $S \leftarrow S'$ } Loop until convergence, i.e. $ V'(s) - V(s) \leq \epsilon \forall s$: For ea transition: { $V'(S_t) \leftarrow V(S_t) + \alpha[R_{t+1} + \gamma V(S_{t+1}) - V(S_t)]$; $V \leftarrow V'$ }
TD Control	Sarsa : On-Policy: Policy eval using TD to approx q_π , Pip using ϵ -greedy. Note: π (behavior might be ϵ -greedy). Loops: { $\pi \leftarrow$ derive from Q ; W π : get (S, A, S', R). Choose action A' given by $\pi(\cdot S')$; $Q(S, A) \leftarrow Q(S, A) + \alpha[R_{t+1} + \gamma Q(S', A') - Q(S, A)]$; $S \leftarrow S'$ }	
	Q-Learning : Off-Policy: Policy eval using TD to approx q_* , Pip using greedy. { ... $Q(S, A) \leftarrow Q(S, A) + \alpha[R_{t+1} + \gamma \max_{a'} Q(S', A') - Q(S, A)]$ }	
Maximization Bias	Q-learning/Sarsa: greedy/ ϵ -greedy policy given curr action. Max over estimated values $\approx$ max value , $\Rightarrow$ +ve bias	
Double Q-Learning	Use 2 action-value fns : Q_1 to estimate value, Q_2 to choose best action: $V(s) = \max_a Q_1(s, \operatorname{argmax}_a Q_2(s, a))$	
n-step Bootstrapping	N-step return w bootstrap: $G_{t:t+n} = R_{t+1} + \gamma R_{t+1} + \dots + \gamma^n V_{t+n-1}(S_{t+n})$ according to policy π n-Step TD Prediction: $V(S_t) \leftarrow V(S_t) + \alpha[G_{t:t+n} - V(S_t)]$. Same for n-Step Sarsa/Q-learning	
Model-based	1) Dist models : $\mathbb{V}_{s',r} s', r, \hat{p}(s', r s, a) \leftarrow \operatorname{Model}(s, a)$. 2) Sample models : produces a possible transition: $s', r \leftarrow \operatorname{Model}(s, a)$	
Planning	1) Background (BG) planning. 2) Decision-time/Online planning: - planning performed at moment of choosing an action	
Model-based BG planning	1) Q-planning : use Q-learning on simulated experiences. (Instead of running in real world, use model to simulate s', r)	
	2) Dyna-Q : (real world + model). Loop: { - $S \leftarrow$ curr state. - $A \leftarrow \epsilon$ -greedy based on Q on state S . - Get real (s, a, r, s') . - Update $Q(S, A)$ using Q-learning based on actual R, S' . - Update <i>Model</i> based on the transition $S, A \rightarrow R, S'$ - Loop n times: { - Randomly sample past (S, A). - Use <i>Model</i> (S, A) to simulate (R, S') . - Update $Q(S, A)$ using Q-learning w R, S' } }	
	Dyna-Q+ : - Suppose transition has actual reward r , & not tried for τ timesteps, then simulated exp reward $= r + \kappa \tau$, for some small κ	
Model-based Decision-time planning	1) Rollout algos : Fix a policy $\pi_r =$ rollout policy. - Aim: improve upon the rollout policy ; not find an optimal policy Estimate $q_{\pi_r}(s, a) \forall a \in \mathcal{A}(s)$ w model simulations: $q_{\pi_r}(s, a) \approx Q(s, a) = \operatorname{Avg}(\operatorname{Returns}(s, a))$. - Policy: $\pi(s) = \operatorname{argmax}_a Q(s, a)$	
	2) MC Tree Search (MCTS) : a) Selection: tree policy till leaf/node w untried actions. b) Expansion: add child node w 1 untried action c) Simulation/rollout: From new node: select actions 1st by tree policy & beyond tree by rollout policy: $Q(s, a) = \operatorname{Avg}(\operatorname{Returns}(s, a))$; $\pi(a s) \leftarrow \epsilon$ -greedy of Q . d) Backup: backup the return generated by the simulated trajectory until the 1st action taken Common tree policy = UCT (Upper Confidence Bound on Trees): $UCT(s, a) = Q(s, a) + c \sqrt{\ln N(s) / N(s, a)}$ (pick highest) $c =$ constant Backup: - Initialize $G =$ rollout return; - For ea step t : { $G \leftarrow G + R_{t+1}$; $Q(S_t, A_t) \leftarrow \operatorname{avg}(\operatorname{Returns}(S_t, A_t))$; $N(S_t, A_t) \leftarrow N(S_t, A_t) + 1$ }	

Value Fn Approximation	Instead of table: use $\hat{v}(s, \mathbf{w}) \approx v_\pi(s)$ for the approx value of state s given weight vector $\mathbf{w} \in \mathbb{R}^d$	
Denote update as $s \mapsto u$, where u = update target: $V(s) \leftarrow V(s) + \alpha[u - V(s)]$	- MC update: $S_t \mapsto G_t$. - TD(0) update: $S_t \mapsto R_{t+1} + \gamma \hat{v}(S_{t+1}, \mathbf{w}_t)$	
Prediction Objective	Let MS value error $\bar{VE}(\mathbf{w}) = \sum_{s \in \mathcal{S}} \mu(s) [v_\pi(s) - \hat{v}(s, \mathbf{w})]^2 \approx \mathbb{E}_\mu [v_\pi(s) - \hat{v}(s, \mathbf{w})]^2$. where $v_\pi(s)$ = true value ; $\mu(s) \geq 0$, $\sum_s \mu(s) = 1$ is state distribution (how much we care about error in each state s) - Often $\mu(s)$ chosen to be fraction of time spent in s. - In on-policy training, this is known as on-policy distribution	
Stochastic Grad Mtds	Suppose observe $S_t \mapsto v_\pi(S_t)$. But dk true value $v_\pi(S_t)$, so use diff target output, $U_t \in \mathbb{R}$, of the t^{th} training example $S_t \mapsto U_t$ Then Minimize $\bar{VE}(\mathbf{w})$ by SGD: $\mathbf{w}_{t+1} = \mathbf{w}_t + \alpha[U_t - \hat{v}(S_t, \mathbf{w}_t)] \nabla \hat{v}(S_t, \mathbf{w}_t)$, where α = +ve step-size	
Gradient MC Prediction	$U_t = G_t$ (MC returns)	Initialize $\mathbf{w} \in \mathbb{R}^d$ arbitrarily. Loop: generate episode w π : for ea timestep: use update eqn
Semi-Gradient Mtds	Approx $v_\pi(S_t)$ with TD(0) return $R_{t+1} + \gamma \hat{v}(S_{t+1}, \mathbf{w}_t)$. But this target/return not independent of $\mathbf{w}_t$ (just ignore) Idea: $U_t = \text{TD}(0)$ return. Algo: Input: Same initialization. Loops $\{-- \mathbf{w} \leftarrow \mathbf{w} + \alpha[R + \gamma \hat{v}(S', \mathbf{w}) - \hat{v}(S, \mathbf{w})] \nabla \hat{v}(S, \mathbf{w}). -- S \leftarrow S'\}$	
Linear Fn Approx	Linear Fns : For ea state s, define $\mathbf{w}$ = feature vector : $\mathbf{x}(s) \in \mathbb{R}^d$. The approx value fn, $\hat{v}(s, \mathbf{w}) = \mathbf{w}^T \mathbf{x}(s)$. And $= \nabla \hat{v}(s, \mathbf{w}) = \mathbf{x}(s)$ wrt $\mathbf{w}$ Thrm : Gradient MC $\rightarrow$ global optimum of $\bar{VE}$ if $\alpha \downarrow$ over time. Thrm : Semi-gradient TD(0) converges to local optimum (TD fixed point)	
On-policy control	To improve, need value of selecting other action a. Need estimate q_π . - MC: $S_t, A_t \mapsto G_t$. - TD(0): $S_t, A_t \mapsto R_{t+1} + \gamma \hat{q}(S_{t+1}, A_{t+1}, \mathbf{w}_t)$ General gd update for $\hat{q}$: $\mathbf{w}_{t+1} = \mathbf{w}_t + \alpha[U_t - \hat{q}(S_t, A_t, \mathbf{w}_t)] \nabla \hat{q}(S_t, A_t, \mathbf{w}_t)$ Semi-Grad Sarsa . : Policy eval using semi-grad TD(0) to approx q_π . Policy improvement using ϵ -greedy. Loops: $\{ \text{Using } \pi: \text{get } (S, A, S', R); -- \text{Choose action } A' \text{ as fn of } \hat{q}(S', \cdot, \mathbf{w}) \text{ (}\epsilon\text{-greedy). } -- \mathbf{w} \leftarrow \mathbf{w} + \alpha[R + \gamma \hat{q}(S', A', \mathbf{w}) - \hat{q}(S, A, \mathbf{w})] \nabla \hat{q}(S, A, \mathbf{w}). S, A \leftarrow S', A' \}$	
Off-policy control	Semi-Grad TD(0) Off-Policy Prediction . Idea: approx $v_\pi(S_t)$ using TD(0) return $R_{t+1} + \gamma \hat{v}(S_{t+1}, \mathbf{w}_t)$. Algo: Loops $\{-- \text{Using } \pi: \text{get } (S, A, S', R); -- \mathbf{w} \leftarrow \mathbf{w} + \alpha \rho [R + \gamma \hat{v}(S', \mathbf{w}) - \hat{v}(S, \mathbf{w})] \nabla \hat{v}(S, \mathbf{w}). -- S \leftarrow S'\}$ & $\rho = \frac{\pi(A S)}{b(A S)}$ (importance sampling ratio) Semi-Grad Expected Sarsa . Idea (GPI): Policy eval using semi-grad TD(0) to approx q_π . Policy improvement wrt $\hat{q} \approx q_\pi$. Initialize $\mathbf{w}$ Loops: $\{-- b \leftarrow \text{derive from } \hat{q} \text{ (e.g. } \epsilon\text{-greedy). } -- \pi \leftarrow \text{derive from } \hat{q} \text{ (e.g. greedy). } -- \text{Using } b: \text{get } (S, A, S', R); -- \mathbf{w} \leftarrow \mathbf{w} + \alpha[R + \gamma \sum_a \pi(a S') \hat{q}(S', a, \mathbf{w}) - \hat{q}(S, A, \mathbf{w})] \nabla \hat{q}(S, A, \mathbf{w})$. Recall $(v_\pi(s) = \sum_a \pi(a s) q(s, a))$. $S \leftarrow S'$ Semi-Grad Q-Learning . Idea (GPI): Policy eval using semi-grad TD(0) to approx q_* . Policy improvement using greedy wrt $\hat{q} \approx q_*$ Same initialization & loops except: $-- \mathbf{w} \leftarrow \mathbf{w} + \alpha \left[R + \gamma \max_a \hat{q}(S', a, \mathbf{w}) - \hat{q}(S, A, \mathbf{w}) \right] \nabla \hat{q}(S, A, \mathbf{w})$. Recall $(v_*(s) = \max_a q_*(s, a))$. $S \leftarrow S'$ Deadly Triad (divergence): 1) Fn approx: large state space. 2) Bootstrapping (TD): opp of long episodes (MC mtds). 3) Off-policy training.	
Modern approach	1) Target Network. - Semi-grad mtds $\neq$ true grad mtds: freeze weight vector used in update target/ target network = $\mathbf{w}_{tgt}$ - Update $\mathbf{w}_{tgt} \leftarrow \mathbf{w}_t$ periodically w latest weight vector of network. Then $\mathbf{w}_{t+1} = \mathbf{w}_t + \alpha[R_{t+1} + \gamma \hat{v}(S_{t+1}, \mathbf{w}_{tgt}) - \hat{v}(S_t, \mathbf{w}_t)] \nabla \hat{v}(S_t, \mathbf{w}_t)$ 2) Experience Replay: Let $\mathcal{D}$ = list for storing transitions = replay memory . - Sample mini-batch transitions uniformly from $\mathcal{D}$. - Train policy/value fn $\pi, \hat{v} \leftarrow \{(S_i, A_i, R_i, S'_i)\}$ 3) Q Network: CNN and Feedforward NN Deep Q Learning (DQN): combines all 3. Algo: (k = param). Loops: $\{-- \text{Sample } N \text{ mini-batch transitions uniformly from } \mathcal{D}$ $-- \mathbf{w} \leftarrow \mathbf{w} + \alpha \frac{1}{N} \sum_{i=1}^N [R_i + \gamma \max_a \hat{q}(S'_i, a, \mathbf{w}_{tgt}) - \hat{q}(S_i, A_i, \mathbf{w})] \nabla \hat{q}(S_i, A_i, \mathbf{w}). -- S \leftarrow S'.$ -- Update $\mathbf{w}_{tgt} \leftarrow \mathbf{w}$ every k steps $\}$	

Policy approx	$\pi(a s, \boldsymbol{\theta})$. - Maximize perf $J(\boldsymbol{\theta})$. Grad ascent : $\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t + \alpha \nabla J(\boldsymbol{\theta}_t)$, where $\nabla J(\boldsymbol{\theta}_t)$ = stochastic estimate whose expectation $\approx \nabla J(\boldsymbol{\theta})$ For discrete action: numerical preferences (softmax in action preferences) . For cts: prob dist. - E.g. normal dist, & learn mean & s.d Pros: can be deterministic policy, unlike ϵ -greedy. - simpler to estimate (than calculate q). - Better for large action spaces	
Policy grad (PG) mtds	Assume episodic , start state s_0 : Let $J(\boldsymbol{\theta}) = \mathbb{E}_{\pi_\theta} [G_t S_t = s_0] = v_{\pi_\theta}(s_0)$, where v_{π_θ} = true value fn for policy π_θ parameterized by $\boldsymbol{\theta}$ Policy Grad Thrm : Assume no discount factor. $\nabla J(\boldsymbol{\theta}) \propto \sum_s \mu(s) \sum_a q_\pi(s, a) \nabla \pi(a, s \boldsymbol{\theta})$, where $\mu(s)$ = state dist, i.e. on policy dist under π $\nabla J(\boldsymbol{\theta}) \propto \mathbb{E}_\pi \left[G_t \frac{\nabla \pi(A_t S_t, \boldsymbol{\theta})}{\pi(A_t S_t, \boldsymbol{\theta})} \right]$. Grad ascent using MC estimate : $\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t + \alpha G_t \frac{\nabla \pi(A_t S_t, \boldsymbol{\theta}_t)}{\pi(A_t S_t, \boldsymbol{\theta}_t)} = \boldsymbol{\theta}_t + \alpha \gamma^t G_t \nabla \ln \pi(A_t S_t, \boldsymbol{\theta}_t)$ ($\because \nabla \ln x = \frac{\nabla x}{x}$) REINFORCE: MC Policy Gradient : Loops: $\{ - G \leftarrow \sum_{k=t+1}^T \gamma^{k-t-1} R_k; - \boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \alpha G \nabla \ln \pi(A_t S_t, \boldsymbol{\theta}) \}$ Thrm : Expected update of REINFORCE over an episode is in same dir ⁿ as performance grad $\nabla J(\boldsymbol{\theta})$ (by construction) REINFORCE: Variance. $Var(X) = \mathbb{E}[(X - \mathbb{E}[X])^2]$. For REINFORCE, $X = G \nabla \ln \pi(A_t S_t, \boldsymbol{\theta})$ = high variance = learning slow & unstable Generalized PG thrm w qty baseline $b(s)$: $\nabla J(\boldsymbol{\theta}) \propto \sum_s \mu(s) \sum_a [q_\pi(s, a) - b(s)] \nabla \pi(a s, \boldsymbol{\theta}) = \sum_s \mu(s) \sum_a q_\pi(s, a) \nabla \pi(a s, \boldsymbol{\theta})$. (Exp same) 1) Constant $b(S_t) = c$ = avg return over epi 2) State-value fn $b(S_t) = v_\pi(S_t) \approx \hat{v}(S_t, \mathbf{w})$ (learned using MC/TD). - near-optimal REINFORCE w Baseline : initial steps same until $\{ - G \leftarrow \sum_{k=t+1}^T \gamma^{k-t-1} R_k; - \delta \leftarrow G - b(s); - \boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \alpha \gamma^t \delta \nabla \ln \pi(A_t S_t, \boldsymbol{\theta}) \}$ REINFORCE w Value Baseline : $\{ - G \leftarrow \dots; - \delta \leftarrow G - \hat{v}(S_t, \mathbf{w}); - \mathbf{w} \leftarrow \mathbf{w} + \alpha^w \delta \nabla \hat{v}(S_t, \mathbf{w}); - \boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \alpha^\theta \gamma^t \delta \nabla \ln \pi(A_t S_t, \boldsymbol{\theta}) \}$ Entropy of policy at $S_t = H(\pi(\cdot S_t, \boldsymbol{\theta})) = - \sum_a \pi(a S_t, \boldsymbol{\theta}) \log \pi(a S_t, \boldsymbol{\theta})$. For exploration: $J_{reg}(\boldsymbol{\theta}) = J(\boldsymbol{\theta}) + \beta \mathbb{E}_\pi [H(\pi(\cdot S_t, \boldsymbol{\theta}))]$	
Actor-critic mtds	$\nabla J(\boldsymbol{\theta}) \propto \mathbb{E}_\pi \left[R_{t+1} + \gamma v_\pi(S_{t+1}) \frac{\nabla \pi(A_t S_t, \boldsymbol{\theta})}{\pi(A_t S_t, \boldsymbol{\theta})} \right]$. Approx expected return using TD return $\approx R_{t+1} + \gamma \hat{v}(S_{t+1}, \mathbf{w})$. - Value fn = critic , π = actor 1-step Actor-Critic w Value Baseline: Loops: - Initialize S . - $I \leftarrow 1$ - Loop for ea step of episode: $\{-- \text{Take action } A \sim \pi(\cdot S, \boldsymbol{\theta}), \text{ observe } R, S'; -- \delta \leftarrow R + \gamma \hat{v}(S', \mathbf{w}) - \hat{v}(S, \mathbf{w}); -- \mathbf{w} \leftarrow \mathbf{w} + \alpha^w \delta \nabla \hat{v}(S, \mathbf{w}); -- \boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \alpha^\theta \gamma I \delta \nabla \ln \pi(A S, \boldsymbol{\theta}); -- S, I \leftarrow S', \gamma I \}$ Advantage $= A_\pi(s, a) = q_\pi(s, a) - v_\pi(s)$. Suppose $S_t, A_t, S_{t+1}, R_{t+1}$: $A_\pi(S_t, A_t) = R_{t+1} + \gamma v_\pi(S_{t+1}) - v_\pi(S_t) = \delta_t$. (i.e. optimize A_π) Generalized Advantage Estimation (GAE) : $\approx$ exponentially weighted sum of future advantages: $A_\pi^{GAE(\lambda)}(S_t, A_t) = \sum_{i=0}^T (\gamma \lambda)^i \delta_{t+i}$ $\lambda = 0 \rightarrow$ std/1-step adv/TD: $= \delta_t = A_\pi(S_t, A_t)$. $\lambda = 1 \rightarrow$ n-step adv/MC: $A_\pi^{GAE(1)}(S_t, A_t) = R_{t+1} + \gamma R_{t+2} + \dots + \gamma^T v_\pi(S_T) - v_\pi(S_t)$ 1) A2C: Synchronous : multiple env in parallel, batched update 2) A3C: Asynchronous : updates happen at diff times	

Constrained policy optimizat ⁿ mtds	Issue: PG update: - Grad specifies dir ⁿ , α controls step size. - Small change in param $\rightarrow$ large change in π	
Surrogate Objective for Policy Optimization	Perf diff lemma : $J(\pi') = J(\pi) + \frac{1}{1-\gamma} \mathbb{E}_{s \sim d_{\pi'}} [\mathbb{E}_{a \sim \pi'} [A_\pi(s, a)]]$, where $d_{\pi'}(s) = (1-\gamma) \sum_t \gamma^t Pr\{S_t = s \pi'\}$. Approx $d_{\pi'}$ using d_π . Approx w importance sampling: $L(\pi') = J(\pi) + \frac{1}{1-\gamma} \mathbb{E}_{s \sim d_\pi} \left[\mathbb{E}_{a \sim \pi} \left[\frac{\pi'(a s)}{\pi(a s)} A_\pi(s, a) \right] \right]$. - Can remove constants $J(\pi)$ and $\frac{1}{1-\gamma}$ Renaming $\pi \rightarrow \pi_{old} = \pi(\cdot \cdot, \boldsymbol{\theta}_{old})$, $\pi' \rightarrow \pi(\cdot \cdot, \boldsymbol{\theta})$ gives surrogate objective : $L(\boldsymbol{\theta}) = \mathbb{E}_{\pi_{old}} \left[\frac{\pi(a s, \boldsymbol{\theta})}{\pi(a s, \boldsymbol{\theta}_{old})} A_{\pi_{old}}(s, a) \right]$ (valid if $\pi \approx \pi_{old}$)	
Trust Region in Policy Optimizat ⁿ	KL divergence: $\mathbb{E}_{\pi_{old}} [D_{KL}(\pi(\cdot s, \boldsymbol{\theta}_{old}) \ \pi(\cdot s, \boldsymbol{\theta}))] \leq \epsilon$. TR = neighborhood ard curr π whr 1st-order $\approx$ is valid	
Trust Region Policy Optimizatⁿ (TRPO)	$\max_{\boldsymbol{\theta}} L(\boldsymbol{\theta}) = \mathbb{E}_{\pi_{old}} \left[\frac{\pi(a s, \boldsymbol{\theta})}{\pi(a s, \boldsymbol{\theta}_{old})} A_{\pi_{old}}(s, a) \right]$, subject to $\mathbb{E}_{\pi_{old}} [D_{KL}(\pi(\cdot s, \boldsymbol{\theta}_{old}) \ \pi(\cdot s, \boldsymbol{\theta}))] \leq \epsilon$	
TRPO Analysis	Thrm (Monotonic impr lower bound): Let $\epsilon = \max_s \left \mathbb{E}_{a \sim \pi} \left[\frac{\pi'(a s)}{\pi(a s)} A_\pi(s, a) \right] \right $, $\delta = \max_s D_{TV}(\pi'(\cdot s), \pi(\cdot s))$: $J(\pi') \geq L(\pi') - \frac{4\epsilon\gamma}{(1-\gamma)^2} \delta^2$ KL vs TV: (easier to compute). Pinsker's inequality : For prob dists p and q on same measurable space, $D_{TV}(p, q)^2 \leq \frac{1}{2} D_{KL}(p \ q)$	
TRPO algo	Loops: $\{ - \boldsymbol{\theta}_{old} \leftarrow \boldsymbol{\theta}; - \text{Sample trajectories using } \pi(\cdot \cdot, \boldsymbol{\theta}_{old}); - \text{Update baselines } v(\cdot \mathbf{w}) \text{ w trajectories}; - \text{Find adv estimate } A_{\pi_{old}}(s, a); - \text{Find update proposal } \Delta \boldsymbol{\theta} \text{ by } \approx \text{following natural grad dir}^n \text{ \& scale to match TR radius}; - \text{Find largest step fraction } \beta \in \{1, 0.5, \dots\} \text{ s.t. } \{-- \boldsymbol{\theta}_{cand} \leftarrow \boldsymbol{\theta}_{old} + \beta \Delta \boldsymbol{\theta}; -- D_{KL}(\pi(\cdot s, \boldsymbol{\theta}_{old}) \ \pi(\cdot s, \boldsymbol{\theta})) \leq \epsilon \text{ \& } L(\boldsymbol{\theta}_{cand}) \geq L(\boldsymbol{\theta}_{old}) \}; - \boldsymbol{\theta} \leftarrow \boldsymbol{\theta}_{cand} \text{ or don't update if no } \beta \text{ satisfies above } \}$	

Problem w TRPO	- Complex. - Expensive finding update proposal $\Delta\theta$ (natural grad & line search). - Constraint satisfaction brittle in practice
Proximal Policy Optimization (PPO):	Use surrogate objective of TRPO but replace KL divergence w clipped importance sampling ratio $L^{PPO}(\theta) = \mathbb{E}_{\pi_{old}} \left[\min \left(\rho(\theta) A_{\pi_{old}}(s, a), \text{clip}(\rho(\theta), 1 - \epsilon, 1 + \epsilon) A_{\pi_{old}}(s, a) \right) \right], \text{ where } \epsilon = \text{hyperparam, and } \rho(\theta) = \frac{\pi(a s, \theta)}{\pi(a s, \theta_{old})}$

POMDPs = MDPs + Partial Observability	POMDP M $\triangleq$ (S, A, E, T, O, R) E = evidence /observations, (can have discount factor γ), transition fn $T: S \times A \times S \rightarrow [0,1]$ observation fn /sensor model $O: S \times E \rightarrow [0,1]$ s.t. $\forall s \in S, \forall e \in E: \sum_{e \in E} O(s, e) = \sum_{e \in E} P(e s) = 1$. - Markov property . P(A B) = P(B A) * P(A) / P(B). Posterior = Observation fn * Prior / Normalizing constant. - Or P(A B) = $\alpha P(B A)P(A)$
Belief States	b(s) = prob it believes it is in state s. If b(s) = curr belief, executes a & receives evidence e', then updated belief given by filtering : $b'(s') = \alpha P(e' s') \sum_s P(s' s, a) b(s)$, where α = normalizing constant. Filtering fn: $b' = \text{FORWARD}(b, a, e')$ aka Bayes Filter . - Similar to T
Decision Making	Optimal $\pi = \text{map } \pi^*(b)$ from belief to action : Given curr belief b, run $a = \pi^*(b)$. Percept e'. Set belief $b' = \text{FORWARD}(b, a, e')$ and repeat Percept: $P(e' a, b) = \sum_{s'} P(e' s') \sum_s P(s' s, a) b(s)$. Transition: $P(b' a, b) = \sum_{e'} P(b' e', a, b) \sum_{s'} P(e' s') \sum_s P(s' s, a) b(s)$ where $P(b' e', a, b) = 1$ if $b' = \text{FORWARD}(b, a, e')$ & 0 otherwise. Expected reward: $\rho(b, a) = \sum_s b(s) \sum_{s'} P(s' s, a) R(s, a, s')$
POMDP into an MDP	P(b' b, a) & $\rho(b, a) \Rightarrow$ observable MDP in belief states space = solving cts, infinite state MDP in belief-state space (infinite beliefs) Optimal policy for MDP $\pi^*(b)$ = Optimal policy for original POMDP.
VI for POMDPs	π = conditional plan (conditioned on observations made so far). If $\alpha_1 \rightarrow a_1$, if $\alpha_2 \rightarrow$ take a_2 ("seq-form"/"treeplex")
Utility fn for Belief state	Belief state = prob dist over states . Ea utility value in POMDP = fn of an entire prob distribution ($V^n(b)$) (piecewise linear fns) Utility of executing conditional plan p in physical state s = $\alpha_p(s)$, where α_p = alpha vector . Expected utility of executing p in belief state b : $U_p(b) = \sum_s b(s) \alpha_p(s)$ or $b \cdot \alpha_p$. Linear fn of b corresponds to a hyperplane in belief space At any b, optimal π = conditional plan w highest utility: $U^{\pi^*}(b) = U(b) = \max_p b \cdot \alpha_p$ (max of hyperplanes $\Rightarrow$ piecewise linear & convex) For finite depth, there are finite num of conditional plans: A actions, E observations, then $ A ^{O(E ^{d-1})}$ distinct depth-d plans 1) suboptimal and dominated plans . 2) undominated plans (ea optimal in a specific region)
VI in POMDP	Let p be a depth-d conditional plan w initial action a followed by depth (d - 1) subplans $p \cdot e'$ for percept e' $\alpha_p(s) = \sum_{s'} P(s' s, a) [R(s, a, s') + \gamma \sum_{e'} P(e' s') \alpha_{p \cdot e'}(s')]$. Given utility fn: find which hyperplane is optimal at any b POMDP VI: - maintains collection of undominated plans {p} w their hyperplanes. - Cf MDP VI computes 1 utility num, U(s) for ea state s $ A n^{ E }$ conditional plans generated at level d + 1 before elimination, where n = num of conditional plans at level d

Imperfect information	Backward induction (minimax search). - Need to be systematic. - Known as Extensive-Form Games
Nash Equilibrium (NE)	Best responses: if other player fixes choice of action (possibly randomized), how should I play? - BRs are typically set valued Pure strategy NE = pair of actions (a_1, a_2) s.t. neither player is incentivized to unilateral deviate , whr $a_1 \in BR_1(a_2)$ & $a_2 \in BR_2(a_1)$ NE captures idea of stability . Pure NE: "local maxima". - Pure NE may not exist. - $u_1(a_1, a_2)$ = P1 payoff when (a_1, a_2) played Mixed strategy NE = pair of dists over actions (x, y) s.t. $x \in \Delta_1, y \in \Delta_2$ and neither player is incentivized to unilaterally deviate. - Probability Simplex: $\Delta_i = \{x \in \mathbb{R}_+^{ \mathcal{A}_i } \mid \sum_j x_j = 1\}$. - $x \in BR_1(y)$ and $y \in BR_2(x)$. Thrm: A mixed NE always exists.
Elimination of Dominated Strategies	Simplify games by ignoring strategies that are always worse than another strategy for a player. - Weak dominance = $\geq$ Domination by mixed strategies: dominated strategy will never be part of NE: can ignore
Mtd to find NE	1) Iterative eliminate dominated strategies (don't always work). 2) Brute force (get all NE): - support & vertex enumerat ⁿ
Remarks	- Multiple NE can occur, but payoff can be diff. Equilibrium selection = choosing "preferred" NE. Not exchangeable (vs 0-sum)
NE in matrix form	Battle of the sexes $\rightarrow A = \begin{pmatrix} 8 & 0 \\ 0 & 4 \end{pmatrix}$ and $B = \begin{pmatrix} 4 & 0 \\ 0 & 8 \end{pmatrix}$, where $n = \mathcal{A}_1 $ and $m = \mathcal{A}_2 $, $A, B \in \mathbb{R}^{n \times m}$, A_{ij}, B_{ij} : payoff given actions i, j P1's strategy $x \in \Delta_n, x \in \mathbb{R}^n$ (prob dist). P2's strategy $y \in \Delta_m, y \in \mathbb{R}^m$. $x^T A y = \sum_i \sum_j x_i \cdot y_j \cdot A_{ij}$ = Row player's expected payoff under x, y. Ay = i th row: How much row player/player 1 would have gotten if by playing the i th action assuming other player plays y NE is a pair s.t.: $x^T A y \geq x'^T A y \forall x' \in \Delta_n$ & $x^T B y \geq x'^T B y \forall y' \in \Delta_m \equiv x^T A y \geq e_i A y \forall i \in [n]$ & $x^T B y \geq x^T B e_j \forall j \in [m]$, e = basis vector
Other games	P1, P2 payoffs = $x^T A y, x^T B y$. Full cooperative (A = B). Zero-sum games: A = -B. - Fully competitive, - P1 (P2) min (max) $x^T A y$
Mini-max Thrm (0-sum game)	Outcome f(x, y) = P1 x, P2 y. $\min_{x \in X} \max_{y \in Y} f(x, y)$: fix x, then find max y (choose x that will min under worst case max) Min-max inequality (always holds): $\min_{x \in X} \max_{y \in Y} f(x, y) \geq \max_{y \in Y} \min_{x \in X} f(x, y)$ Minimax Thrm: If f(x, y) cts, - X, Y compact (closed, bounded) - x convex - y concave: $\min_{x \in X} \max_{y \in Y} f(x, y) = \max_{y \in Y} \min_{x \in X} f(x, y)$ Saddle Points: grad = 0, minimizes x for fixed y (vice versa). - Hessian (if twice differentiable) has at least 1 +ve and -ve eigenvalue $f = x^T A y, X = \Delta_n, Y = \Delta_m$. - f = bilinear (linear in x & y = convex & concave). $\min_{x \in \Delta_n} \max_{y \in \Delta_m} f(x, y) = \max_{y \in \Delta_m} \min_{x \in \Delta_n} f(x, y)$, where $\min_{x \in \Delta_n} = x^*$ and $\max_{y \in \Delta_m} = y^*$. Then: $\min_x x^T A y^* = x^{*T} A y^* = \max_y x^{*T} A y$, where $x^{*T} A y^*$ = expected payoff of the game (unique)
Nice properties of 2P zero-sum games	Unique value $\rightarrow$ no need for equilibrium selection. - Exchangeable: if (x, y) and (x', y') then (x, y') also Nash Set of NE form a polytope for ea player. - Adding additional actions cannot hurt player (not for general-sum games)

Fictitious play	-Simulate 2 players playing against e.o. - BR to empirical dist of past opponent actions. - Empirical dist converges to Nash.
Quality of Equilibrium	(x_0, y_0) : Saddle Point Residual = P2 regret + P1 regret = $\left(\max_y x_0^T A y - x_0^T A y_0 \right) + \left(x_0^T A y_0 - \min_x x^T A y_0 \right) = \max_y x_0^T A y - \min_x x^T A y_0$ Approx Equilibrium: (x_0, y_0) is an ϵ -approx Nash (ϵ -Nash) if $\min_{x \in \Delta_n} x^T A y_0 \geq x_0^T A y_0 - \epsilon$ AND $\max_{y \in \Delta_m} x_0^T A y \leq x_0^T A y_0 + \epsilon$ - A NE itself is a 0-approximate NE. - Saddle point residual = $\epsilon \Rightarrow \epsilon$ -approximate NE Suppose $\min_{x \in \Delta_n} x^T A y_0 - x_0^T A y_0 = \epsilon_1$ and $\max_{y \in \Delta_m} x_0^T A y - x_0^T A y_0 = \epsilon_2$. Then $\text{SPR} = \epsilon_1 + \epsilon_2 \geq \max(\epsilon_1, \epsilon_2)$
No-Regret Learning	Set no-regret learners against e.o. vs Fictitious play (Follow-the-Leader) - NRL $\Rightarrow$ converge to Nash (for 2P 0-sum games)
Basic Setting	$X \in \Delta_n \subseteq \mathbb{R}^n$. At ea iteration t: - learner chooses $x^{(t)} \in \Delta_n$. - observe loss vector $g^{(t)} \in [0,1]^n$. - reward at time $t = \langle x^{(t)}, g^{(t)} \rangle$ where $\langle a, b \rangle = a^T b$. The $g^{(t)}$ are chosen by an adversary (it knows your algo, can "simulate" your choice of $x^{(t)}$; opp of oblivious) How to choose $x^{(t)}$: 1) Best seq in hindsight: $\sum_{t=1}^T \langle x^{(t)}, g^{(t)} \rangle - \min_x \sum_{t=1}^T \langle x, g^{(t)} \rangle$ = what we got - what we could have got (know future) Instead use 2) best fixed strategy in hindsight : $\sum_{t=1}^T \langle x^{(t)}, g^{(t)} \rangle - \min_x \sum_{t=1}^T \langle x, g^{(t)} \rangle$ = regret = R_T Follow-the-leader algo: choose action w lowest loss always: $\text{argmin}_{x \in \{e_1, \dots, e_n\}} \sum_{t=1}^T \langle x, g^{(t)} \rangle$ (is deterministic; exploitable, need randomness)
Hedge (aka multiplicative weights)	For ea action $w_i^{(t)} \in \mathbb{R}$. Play according to: $x^{(t)} = w^{(t)} / \sum_i w_i^{(t)}$. Update: $w_i^{(t+1)} = w_i^{(t)} \exp(-\eta \cdot g_i^{(t)})$, where η = LR. - $w^{(1)} = [1]^n$ Regret bound for Hedge: $R_T \leq \frac{\log(n)}{\eta} + \frac{\eta T}{2}$. Setting $\eta = 1/\sqrt{T}$ gives $O(\log(n) \cdot \sqrt{T})$ regret. - $\text{SPR} \downarrow O(\log(\max(n, m)) \cdot \frac{1}{\sqrt{T}})$
No-regret learning to NE	General framework for self-play: Loop { Nextstrategy(); ObserveUtility() }. - Avg strategies converge to Nash (approx NE) In practice, use another regret minimizer = Regret Matching : - Store regret vector (over all alternative actions)

	- Choose action in proportion to regret (ignore actions w -ve regret). - RM ⁺ permanently adjusts -ve regrets to 0 at ea timestep
Regret Matching for no-regret learners	Maintain $r_i^{(t)}$ = regret of not taking action i: $r_i^{(t)} = \sum_{\tau=1}^t \langle x^{(\tau)}, g^{(\tau)} \rangle - \sum_{\tau=1}^t g_i^{(\tau)}$. In vector: $r^{(t)} = \sum_{\tau=1}^t \langle x^{(\tau)}, g^{(\tau)} \rangle \mathbf{1} - \sum_{\tau=1}^t g^{(\tau)}$ At time t+1, play $x_i^{(t)} = \max(r_i^{(t)}, 0) / \sum_j \max(r_j^{(t)}, 0)$. If all $r_i^{(t)} \leq 0$: play uniformly
RM vs Hedge	Hedge depends on LR. - Another way is to decay LR. - RM free of LR. - RM works well in practice, only requires memory of size n

Extensive-Form Games (EFGs)	Can be solved by minimax search. Factor in info: - States within belong to same player, have same actions (but dk other players state). - Perfect info -> infosets are singletons. - Dotted oval = info: (P2 don't know if P1 get J or K; play same action) Actions are taken at infosets, not states. - Payoffs can depend on states
Perfect Recall	If state h, h' belong to same info: of player i, then all paths from root to h, h' traverse the same seq of infosets & actions of player i From single player's perspective: - every decision point has at most a single parent. - Tree-like structure. - If payoff at leaf actions and probs for dotted lines given, then finding best response tractable via backward induction
Mtd 1: Conversion to normal forms	Cartesian product of all actions at ea info: . If 2 actions, 6 infosets -> $2^6 = 64$ actions Reduced normal form: performing some actions earlier -> some infosets cannot be reached
Mtd 2: Behavioral Strategies	Normal form: randomization is done ex-ante, draw from a dist of "dictionaries", then just follow dict blindly Behavioral strategy: dist over actions locally for ea info: (ea action have diff prob) Kuhn's Thrm : Under perfect recall, space of behavioral strategies and simplex over normal-form strategies is payoff equivalent: - Just consider behavioral strategies. - Smaller in dims (If a actions per info:, strategy is vector of length a * , vs $ a ^{ }$)
Limitation of Mtd 2	Prob of reaching leaf = P1 action probs along path to leaf * P2 action probs along ... * Chance probs along ... - Not bilinear -> cannot write utilities in form $x^T A y$, where x, y are behavioral strategies. - Nonconvex, so not as useful Mtd 1: useful for game solving, but too big. Mtd 2: small, but not as useful for game solving. Mtd 3: best of both
Mtd 3: Seq form & Treeplexes	Use prob of sequences , rather than probs of actions. - Seq already accounts for probs in parent seq (past actions) taken Treeplexes: Natural strategy space for tree-based decision problems. - Sum over all seqs under an info: = parent sequence $\chi = \{x \in \mathbb{R}_+^n \mid Ex = e\}$. Use treeplex as domains: n = num of sequences. $Ex = e$ gives "these sum-of-children=parent" constraints - e is all 0's (for all "non-root" constraints), except for 1 entry, where it sums to be parent seq (which by default is 1) Treeplex is generalization of simplex (treeplex w 1 info: = simplex). - Convex, compact. - Vertices = pure/deterministic strategies - Deterministic strategies in EFGs = finite depth conditional plans Seq form Payoff matrix: each entry = utility of leaf * nature probs (row = P1 seq, col = P2 seq) Across dotted lines (observations), sum rewards. Across solid lines (actions by opponent), take max of rewards
Counter-factual Regret (CFR) Minimizat ⁿ	Put "local" regret minimizers at ea info: . Regret at ea sub-treeplex $\leq$ sum of regret of ea local regret minimizer in its descendant infosets (recursive). $\therefore$ if ea local regret minimizer has sublinear regret $\Rightarrow$ regret over full treeplex also sublinear Can take avg over seq form strategies -> converges to NE in zero-sum games
Immediate CFR (subtree)	Freeze behavioral strategy at every other decision point: - Loss/Reward vector = what we would obtain if all other decision makers played their recommended local strategy. - Assume that player always plays towards that info:
Full CFR (subtree root)	Take any info: and consider the sub-treeplex rooted there : - what we got vs if we change any actions from that info: and its descendants . - Theoretical construct, hard to compute in practice (enumerate vertices)
CFR	Sum of immediate regrets bounds full regret: Lemma 7: $R_{i,full}^{T,+}(I) \leq \sum_{I' \in D(I)} R_{i,imm}^{T,+}(I')$. At root: Thrm 3: $R_i^T \leq \sum_{I \in \mathcal{I}_i} R_{i,imm}^{T,+}(I)$ So can just minimize immediate CFR ($\sum_{I' \in D(I)} R_{i,imm}^{T,+}(I')$) to be sublinear in T. - Use any regret minimizer (e.g. RM+) Work in behavioral strategies and propagate loss (or reward) vector $g^{(t)}$ upwards the tree. Compute value of ea info: v_i At root: get vector $\hat{g}^{(t)}$ that considers the "weighted cumsum" of all a sequence's descendants
Summary of CFR	For ea player: - For regret minimizer in each info: : -- Get next strategy, concat to form behavioral strategy - Convert to seq form (store for averaging later). - Sent $A^T x^{(t)}$, $A y^{(t)}$ to other player For ea player: - Get $g^{(t)}$, compute $\hat{g}^{(t)}$. - For ea info:, observe relevant cells of $\hat{g}^{(t)}$ (observe rewards/losses in the regret minimizer for that info:). Average sequence form strategies converge to Nash

Markov Games	MDP $\equiv (S, A, R, T, \gamma)$. Markov games decomposes actions into >1 player, i.e. $G \equiv (S, A = (A_1, A_2), R, T, \gamma)$. Simultaneous move - Rewards & transitions based on actions of both players. $R(s, a_1, a_2)$ reward at state s & players action (a_1, a_2) . Same $T(s', a_1, a_2, s)$
NE in Markov Games	Intuitive: some strategy (π_1, π_2) s.t. if $u_i(\pi_1, \pi_2) =$ payoff to player i, then if any player unilaterally deviates, they cannot do better Canonical space for π_i : Natural mapping to "behavioral" strategies: - $\pi_i: S \rightarrow \Delta_{A_i}$ = distribution over player i's actions - Pure strategies = $\pi_i(s)$ is deterministic for all states s. - Instead, must allow for randomness
No-SDE games	Turn-taking games can have no stationary deterministic equilibria (SDE). Either randomization is necessary or use non-stationary strategies. - Holds even when transitions are deterministic
2p0s games Bellman updates	Just change max to minimax : Original: Iterate $V_{i+1}^*(s) \leftarrow \max_{a \in A} [R(s, a) + \gamma \sum_{s'} P(s' s, a) V_i^*(s')]$ To something like this: Iterate $V_{i+1}^*(s) \leftarrow \min_{a_1 \in A_1} \max_{a_2 \in A_2} [R(s, a_1, a_2) + \gamma \sum_{s'} P(s' s, a_1, a_2) V_i^*(s')]$ (but need to add stochastic/random) So final: $\min_{x \in \Delta_{A_1}} \max_{y \in \Delta_{A_2}} \sum_{a_1, a_2} x(a_1) y(a_2) [R(s, a_1, a_2) + \gamma \sum_{s'} P(s' s, a_1, a_2) V_i^*(s')]$